

3.3 ppt

3.3 State Machine 与 Checkpoint

序言

让 Agent 中断后，能够接着完成任务。

- 任务状态：现在做到哪里？
- 步骤记录：做过什么，结果怎样？
- Checkpoint：下次恢复需要哪些信息？

三层学习目标

- 原生能力：会使用。
- 扩展接口：会接入。
- 业务规则：会设计、会验证。

一、pi 的原生中断与恢复能力

pi 能恢复会话；业务层还要验证任务现场。

1.1 不写代码，先保存一次会话

【演示 1】启动 pi，并指定会话存档目录：

代码块

```
1 pi --session-dir ./demo-sessions "在当前目录创建 notes.txt，内容写一句话：3.3 demo，然后读给我看。"
```

检查会话目录：

代码块

```
1 ls demo-sessions/  
2 cat demo-sessions/*.jsonl | head -20
```

会话记录保存对话；业务 Checkpoint 保存恢复依据。

1.2 换一个进程，验证它还记不记得

打开一个新终端，使用相同的工作目录和会话目录：

代码块

```
1 pi --session-dir ./demo-sessions -c "我刚才让你创建的文件叫什么名字？"
```

`-c`：续接最近的会话。

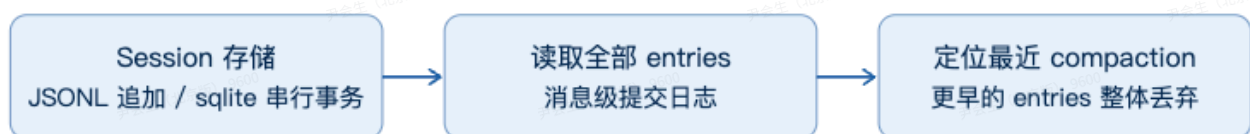
新进程读取历史，继续原有对话。

能回答历史问题，只证明上下文已恢复。

还需检查：文件是否变化、工具是否完成。

1.3 续接时，pi 实际恢复了什么

① 持久化 · 每条消息都是一次独立提交



② 重建 · 从存档回到合法上下文



Pi 恢复的是对话状态：消息链合法、轮次可续（compaction 截断重放，不全量回放）。
工作区动没动、批的是哪份补丁、任务做到第几步——Pi 一概不管，是业务层 Checkpoint 的对账职责。

读取会话记录，重建当前分支上下文。

压缩后使用摘要与保留消息。

底层续跑接口要求消息链有效。

CLI 续接与底层续跑接口属于不同层次。

命令或参数	用途
<code>-c / --continue</code>	续接最近的会话
<code>-r / --resume</code>	打开会话选择器，选择要恢复的会话
<code>--session-id <id></code>	使用精确的项目会话 ID； 不存在时会创建，使用前要核对 ID
<code>--fork <path id></code>	从指定会话派生一个新会话
<code>--session-dir <dir></code>	指定会话存档目录
<code>--no-session</code>	关闭会话持久化

派生会话不会自动复制或还原代码工作区。

二、能继续对话，不等于能恢复现场

恢复之前，先核对记录与真实现场。

2.1 为什么存档和实际代码可能对不上

代码块

```
1  await writeFile(filePath, content.replace(search, replace), "utf8");
2  this.revisionSequence += 1;
```

工具返回后，业务运行层才会在轮次边界保存下一份 Checkpoint。

代码块

```
1  上次存档：代码 r0, patch 尚未完成
2      ↓
3  补丁已经写进 session-policy.ts
4      ↓
```

- 上次存档：补丁尚未执行。
- 实际文件：补丁已经写入。
- 突然退出：新结果尚未保存。
- 直接恢复：可能重复修改、误用旧证据。

没有成功记录，不代表动作没有发生。

2.2 做一次中途打断，观察恢复行为

演示：读取文件 → 中途停止 → 续接任务

代码块

```
1 pi --session-dir ./demo-sessions "逐个读取 src/ 目录下的每个文件，统计总行数，把结果  
  写入 report.md。"
```

重新启动，续接会话：

代码块

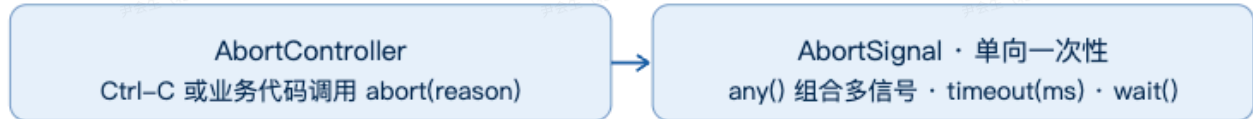
```
1 pi --session-dir ./demo-sessions -c "继续。"
```

- 已读文件是否重复处理？
- 未读文件是否遗漏？
- 最终报告是否完整？

一次续跑成功，不能证明恢复机制可靠。

2.3 中断信号能停下什么

① 触发 · 谁能叫停



② 传播 · agent_loop 里的检查点 (signal 贯穿全程)



③ 收口 · 中止不是抛异常，是把消息链补完整



协作式中止：AbortSignal 只是「请求它停止」，不回滚已经发生的写盘与子进程。
已发生的动作不会消失——这正是业务层 Checkpoint 必须对账的原因。

- **AbortSignal** 传递停止请求。
- 工具需要主动支持取消。
- 已中止的信号不能复用。
- 强制退出可能来不及保存结果。

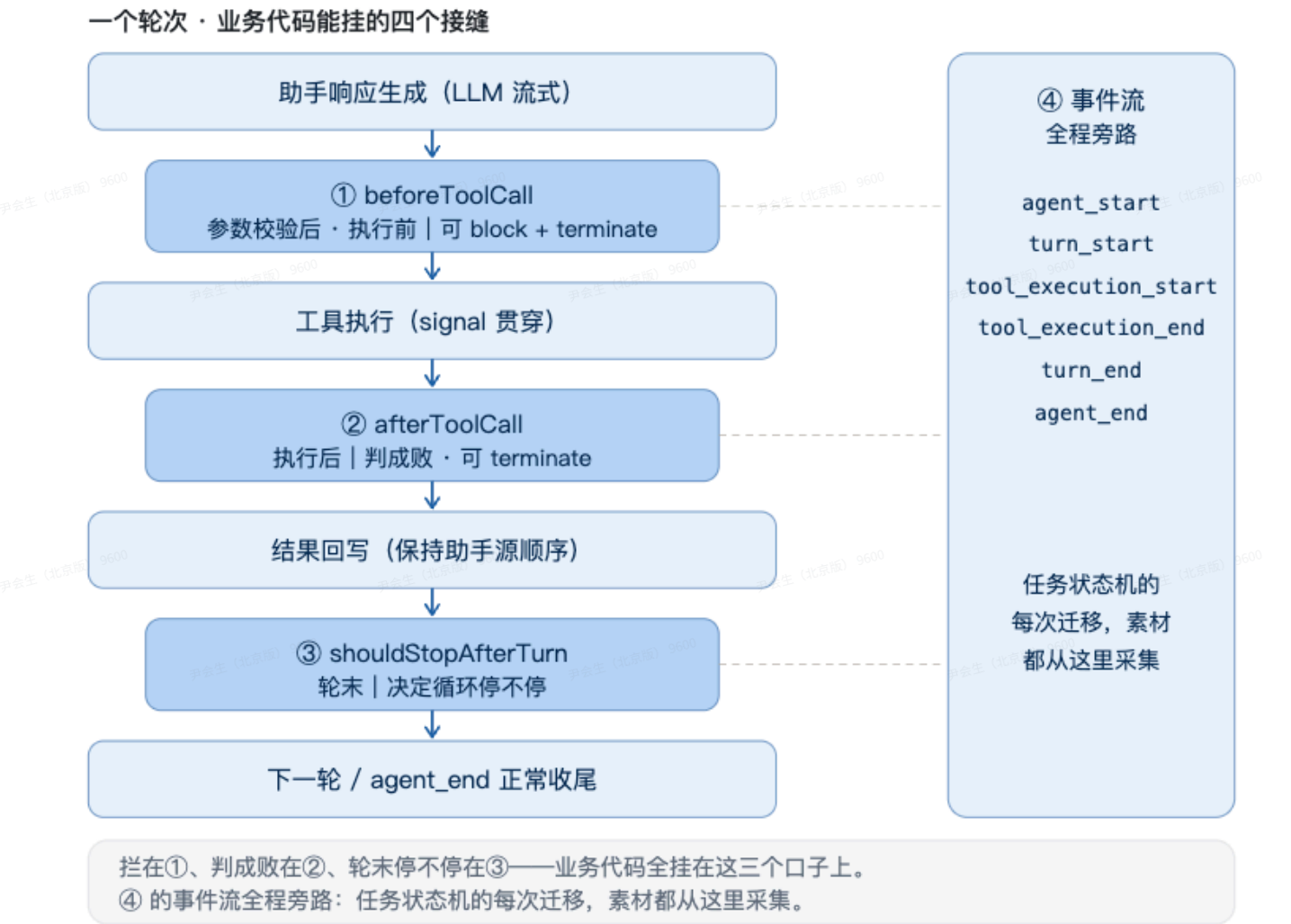
中断不等于回滚：文件修改与子进程需要单独处理。

业务层还需关联：

信息	恢复时用来判断什么
计划与步骤进度	哪些步骤已经完成，下一步准备做什么
工具执行记录与证据	动作是否成功，测试结果是否完整可信
工作区标识与代码版本、摘要	现在的代码是否与保存时一致
待审批动作与人工决定	哪些动作仍在等待确认，已有批准是否还适用

先核对上次留下的状态，确认可以继续，再让模型进入下一轮。

2.4 通过哪些接口接入业务流程



接入位置	业务用途
beforeToolCall	执行前检查与拦截
afterToolCall	记录结果、审计与证据
事件订阅	更新任务与步骤状态
续跑接口	检查通过后继续执行

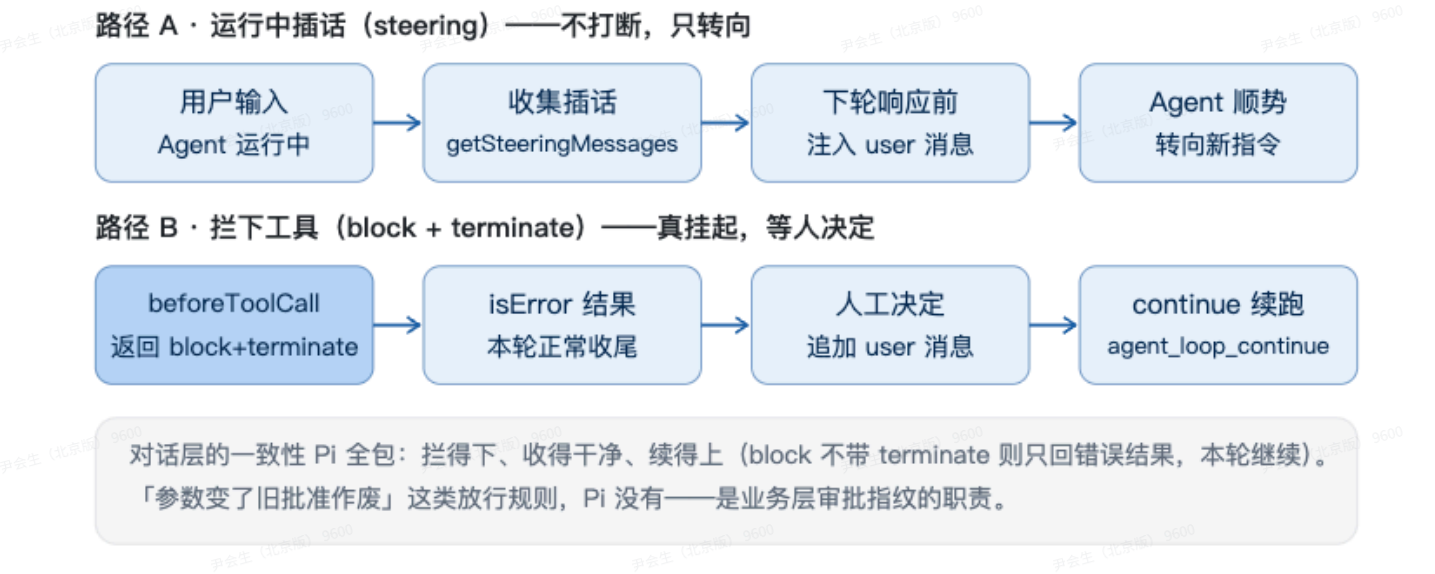
第一， beforeToolCall ：在执行前决定是否放行。

拦截 bash 工具：

```
代码块
1  const agent = new Agent({
2    initialState: {
```

```
3     systemPrompt,
4     model,
5     tools,
6     messages,
7 },
8 // 参数校验之后、真正执行之前被调用；返回 block 就不放行
9 beforeToolCall: async ({ toolCall, args, context }) => {
10     if (toolCall.name === "bash") {
11         return { block: true, reason: "bash is disabled" };
12     }
13 },
14 });
```

区分两种人工介入方式：



“插话”（steering）在这里不是日常聊天里打断别人说话，而是指：Agent 还在跑的时候，你又给它发了一条新消息。这条消息不会立刻中止当前动作，而是被收集起来，等下一轮模型调用之前，作为一条 `user` 消息注入上下文，让 Agent 在下一轮“顺势转向”。

方式	使用场景	需要注意什么
运行中插话（steering）	Agent 正在执行，希望补充要求或调整方向	消息在循环的接收位置进入上下文，不保证立即停止正在运行的动作
暂停等待审批	某个动作执行前必须得到人的批准	业务层保存具体动作并停止推进；批准后重新校验，再恢复执行

第二，`afterToolCall`：根据执行结果补充记录。

这个钩子会收到工具调用、执行结果、错误标志和上下文。我们可以在这里补充审计信息、整理证据，并据此更新步骤记录。

第三，事件订阅：把执行过程转成业务状态。

`agent_start`、`turn_start`、`tool_execution_start`、`tool_execution_end`、`agent_end` 等事件，告诉我们框架运行到了哪里。业务层再把这些事件关联到具体任务和计划步骤。

第四，续跑接口：在恢复检查通过后继续调用模型。

`runAgentLoopContinue` 或 `agent.continue()` 提供的是继续执行的机制。使用前，需要先还原计划、证据、审批状态和运行计数，并完成工作区校验。

关键边界

- 拦截一次调用，不等于完整审批流程。
- Steering 用于调整方向，不保证立即停止动作。
- 单次 `terminate` 提示，不保证整批工具立即停止。
- `agent_end` 表示循环结束，不代表任务完成。

三、业务层需要补齐哪三样东西

业务对象	解决的问题
任务状态机	现在什么情况，允许做什么
步骤执行记录	做过什么，结果与证据是什么
业务 Checkpoint	保存什么，恢复前检查什么

3.1 任务状态机：记录现在到哪了，规定接下来能做什么

修复登录问题的过程：

第一步，收到任务。

第二步，调用工具，等待结果。

第三步，准备改代码，先等人批准。

第四步，验证通过，结束任务。

状态描述当前情况，迁移规则约束下一步。

状态	放到修代码的过程里理解
created	收到修复任务，还没开始处理
planning	正在建立或整理计划
running	正在分析结果、决定下一步
waiting_tool	已发起工具调用，正在等待结果
tool_failed	某次工具调用失败，需要判断如何处理
waiting_approval	某个动作尚未获批，等待人工决定
completed	修复和约定的验证均已完成
failed	本次任务无法继续，未完成交付
cancelled	用户决定结束本次任务

有两个容易混淆的地方。

第一个：工具失败，是不是整个任务就失败了？

第二个：暂停了，是不是应该记为 cancelled

任务阶段：如 running。

停止原因：如 INTERRUPTED。

谁来修改这些状态？由程序根据实际事件和业务规则修改。

3.2 步骤执行记录：哪些工作已经做过

任务状态告诉我们“现在处于什么情况”，步骤执行记录告诉我们“具体做过什么，结果怎么样，还差什么”。

假设上一节的 Planner 已经把任务分成四步：

步骤	要完成的工作	当前进度
定位问题	找到会话过期判断，复现问题	已完成
修改代码	修复过期时间的边界判断	进行中
执行验证	运行目标测试、边界测试和回归测试	未开始
整理报告	写清根因、改动和验证结果	未开始

第一，记录准备做什么。

第二，记录实际发生了什么。

工具执行后，要保存返回结果。

第三，记录失败后准备怎么办。

假设工具明确返回了“找不到原文”，Agent 决定先重新读取文件，再生成补丁。

第四，记录后续尝试与最终结果。

读完最新文件后，Agent 提交了新的补丁，这次成功了。

换参数、换工具，都不能把前面的失败次数和已经消耗的预算清零。

记录内容	回答的问题
所属步骤与步骤进度	这次操作在完成哪一步？这一步做完了吗？
工具名称、输入参数和调用 ID	具体调用了什么？这次操作的流水号是什么？
工具输出	实际返回了什么结果？
失败原因	为什么没成功？还是根本没有拿到完整结果？
重试与后续尝试	已经试过几次？每次做了什么调整？
下一步决策	接下来准备怎么处理？
完成证据	凭什么说这一步已经完成？

补丁成功，只能说明修改已经应用，不能证明问题已经修好。

维护的是两层信息：

上面是计划步骤和进度，

下面是这个步骤中发生的多次工具调用。

3.3 Checkpoint：把恢复所需的信息保存下来

Checkpoint 是任务存档，为恢复提供依据。

第一，存档里到底要保存什么？

保存的信息	恢复时用来回答什么
任务标识、目标和状态	恢复的是哪个任务？原本要做什么？停在什么阶段？
计划和步骤进度	诊断完成了吗？补丁执行了吗？还剩哪些步骤？
工具执行记录和证据	之前做过什么？结果是什么？依据在哪里？
会话引用或必要的历史消息	怎样让模型接上之前的上下文？
工作区位置、代码版本和文件摘要	应该在哪份代码上继续？代码有没有变化？
待审批动作和人工决定	等待批准的是哪份补丁？是否已经获得批准？
已用轮数、重试次数和预算	之前消耗了多少？还允许继续尝试多少次？

怎样保存

- 完整轮次结束后保存。
- 等待审批前保存具体动作与状态。
- 先写临时文件，再替换正式存档。
- 格式版本检查兼容性，校验和检查损坏。
- 保存失败时停止推进，保留有效旧存档。

代码写入与存档保存之间，仍可能出现差异。

怎样恢复

- 校验存档：格式、字段、完整性。
- 核对现场：工作区、文件变化、结果不明的操作。
- 检查规则：任务状态、审批、剩余预算。

- 4. 还原对象：计划、记录、证据与计数。
- 5. 准备上下文，调用 pi 续跑。

恢复依靠记录重建任务，不会自动恢复调用栈或旧子进程。

四、怎样用最小规格指导开发和验收

先约定正确行为，再让 AI 实现。

4.1 最小规格，要写清楚到什么程度

- 进度不能丢。
- 存档要可信。
- 恢复先检查。
- 审批要有效。
- 失败后能排查。

复用 pi 与已有 Planner，只补充缺失的业务规则。

能力	实现方式
模型调用、工具执行、消息回写、事件流	复用 pi 的现有能力
对话续接、中断信号、工具前后 Hook	通过 pi 提供的接口接入
会话记录持久化	项目已经使用 Session / JsonlSessionRepo 时继续复用，先核对当前接入方式
计划与步骤规则、修复和测试工具	沿用 3.2 的项目实现
任务状态与步骤执行记录	补充业务规则，关联 pi 事件和工具调用
业务 Checkpoint 与恢复装配	补充业务数据的保存、校验与还原
人工接管与失败排查	补充审批规则、待确认动作、独立重跑和结果对照

4.2 怎样把要求交给 AI

- 1. 读项目：确认已有能力、缺口与限制。
- 2. 写规格：明确状态、存档、恢复、审批、排查要求。
- 3. 列计划：对应文件、改动与验收场景。
- 4. 做验证：保留日志、文件差异与测试结果。

交付时区分：已实现、已验证、未覆盖。

代码块

- 1 任务：在上一节的 Coding Agent 上，补齐任务状态、Checkpoint 和人工接管。
- 2 课堂案例：修复登录模块的会话过期边界问题，完成验证后输出 artifacts/login-fix.md。
- 3
- 4 一、先检查现有项目
- 5 阅读已有的计划、工具、运行入口、会话存储、测试，
- 6 以及项目实际使用的 pi 版本和相关接口。
- 7 列出：
- 8 1. 已经具备哪些能力。
- 9 2. 还缺哪些业务逻辑。
- 10 3. 哪些能力当前不支持，需要说明限制。
- 11
- 12 继续复用现有模型接入、pi 的 Loop、工具执行、消息回写、
- 13 中断信号、事件流和已有会话存储。
- 14 沿用上一节的 Planner、修复工具和测试工具。
- 15 不要仅凭接口名称推断行为，要核对实际实现。
- 16
- 17 二、按以下最小规格实现
- 18
- 19 R1：任务和步骤记录
- 20 根据实际执行事件更新任务状态。
- 21 保存计划进度、工具调用 ID、输入输出、失败原因、
- 22 重试记录、下一步决策和完成证据。
- 23 暂停和等待审批不算任务失败。
- 24 工具返回成功，不等于测试通过或整个任务完成。
- 25 本例中，已完成、失败或取消的运行不再续跑。
- 26
- 27 R2：保存恢复所需的信息
- 28 保存任务、计划、步骤记录、证据、会话引用或历史、
- 29 工作区标识、代码版本与摘要、运行计数、
- 30 待审批动作和人工决定。
- 31 默认在完整轮次结束后保存；
- 32 进入等待审批前，保存待确认动作和状态。
- 33 明确存档格式和校验规则。
- 34 保存失败时停止推进，保留最近一份有效存档并报告原因。

R3：先检查，再恢复 先校验存档、工作区和任务状态，再还原业务对象，最后调用 pi 的续跑能力。
区分上次动作是已完成、未开始，还是结果不明。
结果不明的写操作不得自动重放。
无法安全确认时，输出核查信息并停止自动推进。
没有完整测试结果，不能把验证步骤记为完成。
恢复时保留已有轮数、重试次数和预算消耗。

R4：人工审批 修改源码前保存具体补丁，进入等待审批状态。
批准前不能执行，拒绝后不能执行。
等待期间，其他需要审批的写操作也不能绕过检查。
批准只适用于对应动作。
补丁参数或相关文件内容变化后，重新检查授权。
不能因为进程重启，就默认获得批准。

R5：失败排查
允许用原任务输入，在对应初始代码的独立工作区重新执行。
保留原运行记录，另存新运行的日志、修改和测试结果。
不默认继承危险操作权限。
不承诺模型会选择相同路径，也不承诺原错误一定再次出现。

本次不实现：
进程内存或调用栈快照、原测试子进程恢复、
任意时刻崩溃后的无损恢复，以及副作用恰好执行一次。
不扩展上下文自动压缩和 Sandbox。
不额外引入事务或调度框架。

三、列出修改计划，再实现

把 R1-R5 对应到现有文件、缺失逻辑和验收场景。
按最小增量修改，不重写已有框架能力。

先准备关键失败场景，再实现对应行为。
不能通过清空记录、重置预算或把状态改成成功来绕过问题。
对影响业务行为的歧义，先说明取舍。

四、按以下场景验收

A1：诊断完成并成功保存后退出进程。
恢复后保留计划、诊断与证据，从后续步骤继续。

A2：补丁已经写入，但下一份存档尚未保存时异常退出。
恢复时检测差异或结果不明，阻止重复写入，输出核查信息。

A3：测试已经启动，但没有完整结果时中断。
恢复后不记为通过。 先确认旧执行已结束或被隔离，再决定是否重新测试。

A4：补丁等待审批。

82 分别验证批准与拒绝： 批准前及拒绝后均不执行，批准后才执行对应补丁。

83

84 A5：批准后，补丁参数或相关文件发生变化。

85 再次执行时，旧授权不能直接放行。

86

87 A6：存档损坏、工作区缺失或任务已经终结。

88 恢复请求被明确拒绝，不启动模型或写工具。

89

90 A7：多次工具失败后暂停。

91 恢复后保留失败记录、运行计数和预算消耗。

92

93 A8：原任务存在失败记录。

94 在独立工作区重跑后，原记录保持不变，

95 能够对照两次运行的日志、修改和测试结果。

96

97 需求与验收对应：

98 R1 → A1、A7

99 R2 → A1、A2、A6

100 R3 → A1、A2、A3、A6、A7

101 R4 → A4、A5

102 R5 → A8

103

104 交付：

105 最小规格、需求与改动及验证的对应表、

106 最小代码改动，以及实际运行证据。

107 分别标明“已实现”“已验证”“未覆盖”。

108 异常退出实验只在临时工作区进行。

109 不能只凭编译通过或模型声称完成，就宣布验收通过。

这份提示词里，R 表示需求，A 表示验收场景。编号只是为了方便对应，不需要记住这些字母。

4.2 场景一：中断之后，能不能接上原来的进度

先用 `--pause-after 6 --auto-approve` 运行。这里自动批准，是为了先单独观察中断恢复，后面再验证审批。

代码块

```
1 npm start -- --workspace /tmp/state-agent-lab --run-id run-lab2 \  
2   --pause-after 6 --auto-approve "修复 demo-app 里会话到期边界判断的错误，补上回归测试并写交付说明"
```

暂停后的摘要如下：

代码块

```
1  停止原因: INTERRUPTED
2  任务状态: running      ← 只是暂停, 没有失败
3  代码版本: r0
4  步骤状态:
5    diag: completed, 证据 1 条  ← 问题已查明
6    patch: pending, 证据 0 条   ← 代码还没改
7  存档: seq=7
8  任务最终结果: 未终结 (可从存档继续)
```

这段输出说明：诊断已经完成，补丁还没执行，任务只是暂停，没有被判为失败。

代码块

```
1  npm start -- --workspace /tmp/state-agent-lab --resume run-lab2 --auto-approve
```

接下来使用项目的恢复入口，观察输出：

代码块

```
1  恢复自: seq=7 | 任务状态=running | 代码版本=r0
2  #29 running -(task_resumed)-> running (restored from checkpoint)
3  #33 waiting_tool -(tool_succeeded)-> running (OK)      ← apply_patch
4  #54 running -(task_completed)-> completed (COMPLETED)
5  代码版本: r1 | diag/patch/verify/report 全部 completed
6  任务最终结果: completed | 完成契约已满足
```

状态显示完成，只是一个结果入口；文件改动和验证记录，才是判断它是否完成的依据。

4.3 场景二：人工批准之前，补丁能否被拦住

去掉自动批准，发起同一个修复任务：

代码块

```
1  WS=$(mktemp -d)
2  echo $WS
3  npm start -- --workspace "$WS" --run-id run-fix "请修复登录模块的会话过期边界问题：
    先定位登录和会话相关代码，复现失败测试，分析根因，只修改必要源码，然后运行目标测试、边界测
```

试和回归测试，最后把根因、改动和验证结果写入 `artifacts/login-fix.md`。"

原稿中的运行在修改源码前停了下来：

代码块

```
1  停止原因: WAITING_APPROVAL
2  任务状态: waiting_approval
3  代码版本: r0
4  状态迁移:
5      #29 running -(approval_requested)-> waiting_approval (apply_patch@call_00_...)
6  计划版本: v1
7  步骤状态:
8      - diagnose: completed, 证据 5 条
9      - fix: in_progress, 证据 0 条
10     - verify: pending, 证据 0 条
11  待人工确认:
12     - apply_patch {"path":"src/auth/session-policy.ts","search":"...", "replace":"..."}
13     - 批准: npm start -- --resume <ID> --approve
14     - 拒绝: npm start -- --resume <ID> --reject
```

这里重点检查三件事。

第一，前面的诊断和证据是否保留。

第二，待确认内容是不是具体补丁。

第三，也是最关键的：文件有没有提前被修改。

选择批准或拒绝。使用时，把 `<运行ID>` 替换成实际 ID：

代码块

```
1  npm start -- --resume <运行ID> --approve
2  # 或者
3  npm start -- --resume <运行ID> --reject
```

输出如下：

代码块

```
1  ===== State Agent: 从存档恢复 =====
2  恢复自: seq=6 | 任务状态=waiting_approval | 代码版本=r0
3  人工决定: 批准 apply_patch
```

```
4 ...
5 #30 waiting_approval -(approval_granted)-> running (approved by operator)
6 #31 running -(tool_started)-> waiting_tool (apply_patch)
7 #47 running -(task_completed)-> completed (COMPLETED)
8 任务最终结果: completed | 完成契约已满足
```

批准路径跑通，只能证明批准路径。拒绝和授权失效，也要分别验证。

4.4 场景三：失败以后，能不能留下可对照的证据

代码块

```
1 WS=/tmp/agent-lab-4-4; rm -rf "$WS"; mkdir -p "$WS"
2 npx tsx scripts/child-run.ts --scenario ac4 --workspace "$WS" --run-id run-bad
3 npx tsx scripts/child-run.ts --scenario ac4 --workspace "$WS" --resume run-bad
  --reject
4 npx tsx scripts/child-run.ts --scenario ac4 --workspace "$WS" --replay run-bad
```

- 准备对应初始代码的独立工作区。
- 使用原任务输入，建立新运行记录。
- 保留原日志、修改与测试结果。
- 对照两次执行路径与结果。
- 危险操作权限不默认继承。

重跑用于对照排查，不保证模型重复原路径。

五、理解底层设计：换一个框架，哪些判断仍然适用

接口可以变化，可靠性判断仍然适用。

5.1 恢复任务，为什么要同时找回对话和业务状态

恢复时要同时准备好两部分：

恢复内容	解决的问题
对话上下文	让模型理解目标和前面的处理过程
业务状态	让程序判断当前进度，以及下一步是否允许执行

模型知道发生过什么，程序确认现在能做什么。

5.2 中断和审批，应该放在执行过程的什么位置

执行位置	需要增加的处理
模型提出补丁之后、工具执行之前	检查参数、权限和审批；不满足条件就拦住
工具返回结果之后	记录实际结果，关联对应步骤和调用 ID
到达约定的保存边界时	保存任务进度、证据和恢复所需的信息
收到停止请求时	停止推进，处理正在执行的操作，记录已知状态
下次启动时	读取存档、核对现场，通过后再继续

先审批再写入，先核验再完成，先检查再恢复。

5.3 为什么发出停止请求，不等于所有操作都停了

必须区分三件事：

1. 用户已经请求停止。
2. Agent 不再发起新的操作。
3. 已经启动的操作确实结束了。
 - `Promise.race` 可以结束等待。
 - 底层操作仍需自行处理取消。
 - 无法及时取消时，等待结束或隔离。

验收要确认操作停止，不能只看页面停止输出。

5.4 停止时留下的记录，要能支持下一次恢复

能够确认的情况	恢复时怎么处理
动作尚未开始	重新检查条件，通过后可以执行
动作已经完成，结果完整	核对现场后，保留已完成进度
动作已经发起，但没有完整结果	先核实实际效果，不能直接重放写操作

- 区分确认失败与结果不明。
- 调用与结果通过 ID 对应。
- 缺失结果不能伪造成成功。
- 强制退出可能没有收尾机会。

5.5 工程师应该掌握到什么程度

- 能检查审批是否真正拦住写操作。
- 能确认中断后工具是否仍在运行。
- 能检查恢复前是否核对现场。
- 能验证失败记录与预算没有清零。
- 能用测试与交付证据判断完成。

能定位缺口、修改规则、设计实验验证。

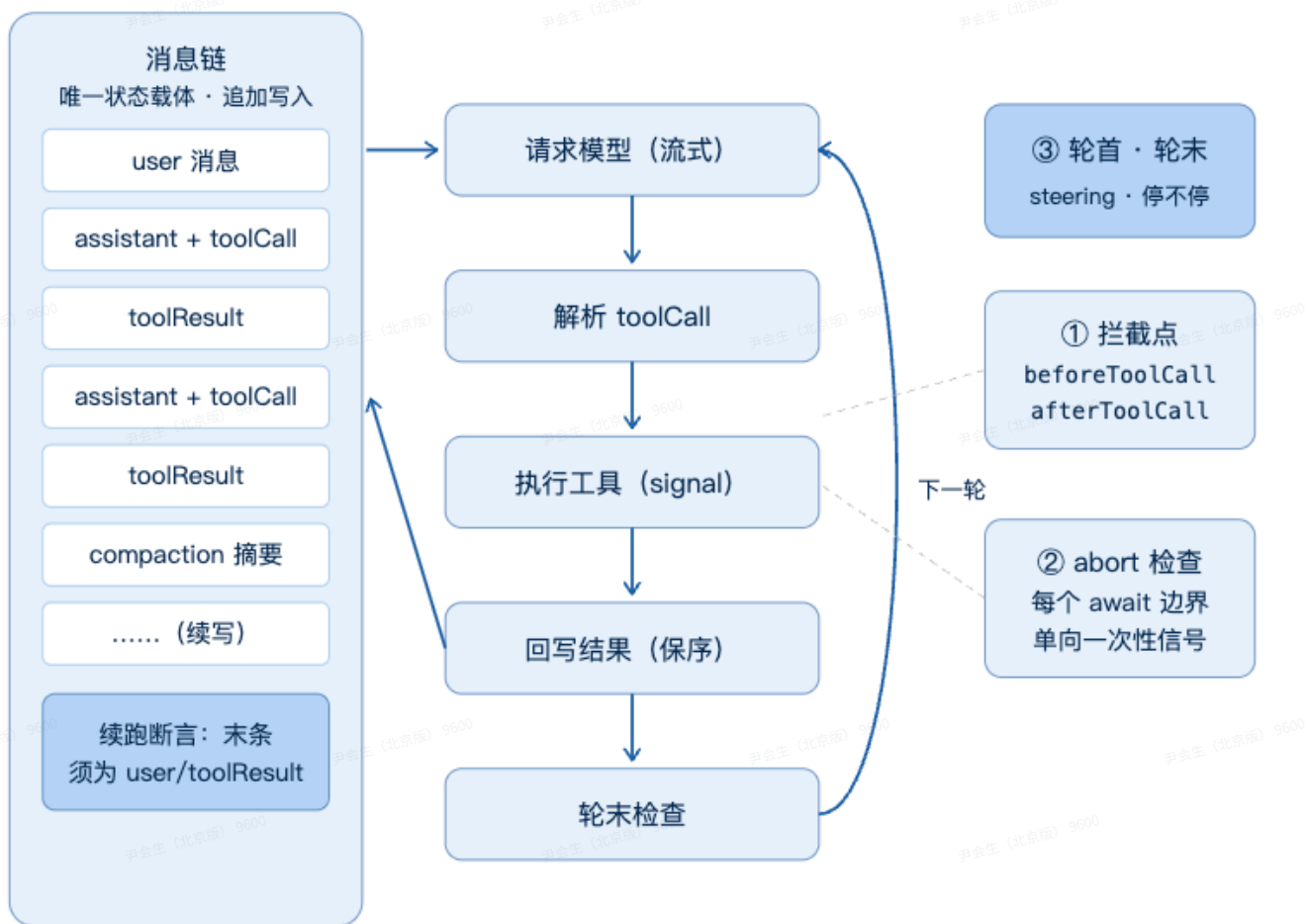
本节小结

任务状态：说明当前阶段，约束允许的动作。

步骤记录：保存执行过程、结果与证据。

Checkpoint：保存恢复依据，支持跨进程继续。

Pi 的设计解剖：一条消息链 + 一个循环 + 三类检查点



停 = 把消息链补成合法状态：中止的调用补 isError 结果，assistant 标 aborted，配对不缺。
续 = 重放 + 断言：从压缩点截断重放，末条必须是 user 或 toolResult。

四条执行规则

- 恢复前，核对现场。
- 结果不明，先查再做。
- 未经批准，不执行修改。
- 宣布完成，必须有证据。

下一节：Sandbox 与执行边界。